

Kiro Features Setup

create a powerful, automated development environment.

Exercise 2.1: MCP Server Configuration

Configure essential MCP servers for AWS development and chatbot c

Configure MCP servers to enhance Kiro's capabilities for our chatbot project

```

Create or update '.kiro/settings/mcp.json' with the following configuration:

'''json
{
  "mcpServers": {
    "aws-docs": {
      "command": "uvx",
      "args": ["aws:aws-documentation-mcp-server@latest"],
      "env": {
        "FASTMCP_LOG_LEVEL": "ERROR"
      },
      "disabled": false,
      "autoApprove": ["search-documentation", "get-service-info"],
      "disabledTools": []
    },
    "github": {
      "command": "uvx",
      "args": ["github-mcp-server@latest"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      },
      "disabled": false,
      "autoApprove": ["list-repositories", "get-repository-info"],
      "disabledTools": []
    }
  }
}
'''

**Step 2: Test MCP Server Connections**
After configuration, test each MCP server:
1. "What are the best practices for AWS Lambda function configuration?"
2. "Show me information about serverless architecture patterns"
3. "What are the latest AWS Bedrock model capabilities?"

Verify that Kiro can access real-time AWS documentation and provide current information.

```

Exercise 2.2: Add

- Add specialized MCP servers for enhanced development capabilities

```
'''json
```

```

mcpServers: {
  "aws-docs": { /* previous config */ },
  "github": { /* previous config */ },
  "npm-registry": {
    "command": "uvx",
    "args": ["npm-mcp-server@latest"],
    "disabled": false,
    "autoApprove": ["search-packages", "get-package-info"],
    "disabledTools": []
  },
  "docker": {
    "command": "uvx",
    "args": ["docker-mcp-server@latest"],
    "disabled": false,
    "autoApprove": ["list-images", "inspect-container"],
    "disabledTools": ["remove-container", "remove-image"]
  },
  "web-search": {
    "command": "uvx",
    "args": ["web-search-mcp-server@latest"],
    "env": {
      "SEARCH_API_KEY": "${SEARCH_API_KEY}"
    },
    "disabled": false,
    "autoApprove": ["search"],
    "disabledTools": []
  }
}
...

**Test Additional Servers**

1. "Find the latest React TypeScript packages for chat interfaces"
2. "Search for AWS CDK best practices for serverless applications"
3. "What are the current Docker base images for Node.js applications?"

```

- Web search capabilities for

- ### Exercise 2.3: Agent Hooks Configuration

****Step 1: Access Hook Configuration****

```

2. Search for "Open Kiro Hook UI"
3. Or find "Agent Hooks" in the explorer view

**Step 2: Create Specification Validation Hook**
Create a hook that validates specification completeness:

**Hook Name**: "Specification Validator"
**Trigger**: Manual button click
**Description**: "Validates that specifications are complete before moving to implementation"
**Actions**:
...

Check that all specification files exist:
- Foundation requirements are complete
- Infrastructure specifications include all AWS resources
- Frontend specifications include UI/UX and technical details
- Backend specifications include API design and business logic
- Security specifications cover all compliance requirements

Generate a validation report with:
- Completed specifications checklist
- Missing or incomplete sections
- Recommendations for improvement
- Next steps for implementation
...

**Step 3: Create Documentation Update Hook**
**Hook Name**: "Documentation Sync"
**Trigger**: File save on specification files
**Description**: "Automatically updates related documentation when specifications change"
**Actions**:
...

When specification files are saved:
1. Update the main README.md with specification status
2. Generate or update architecture diagrams
3. Update the tasks.md file with progress
4. Sync changes to related specification files
5. Validate cross-references between specifications
...

```

Expected Outcome

- Agent Hooks configured for specification validation and documentation sync
- Automated quality checks for specification completeness
- Automatic documentation updates when specifications change
- Enhanced workflow efficiency through automation

Exercise 2.4: Agent Steering Documentation

Create Agent Steering files to

- ```

Create comprehensive Agent Steering documentation in the `./kiro/steering/` dir

File 1: `./kiro/steering/project-standards.md`
---markdown

inclusion: always

```

```
MyCoolChatbot-KiroSpec Project Standards
```

```
Coding Standards
- Use TypeScript for all JavaScript code
```

```

- Use AWS CLI for infrastructure as code
- Implement comprehensive error handling
- Include detailed JSDoc comments

AWS Best Practices
- Apply principle of least privilege for IAM
- Use AWS managed services when possible
- Implement proper logging and monitoring
- Follow AWS Well-Architected Framework
- Optimize for cost and performance

Documentation Requirements
- All specifications must include acceptance criteria
- Code must include comprehensive comments
- API endpoints must have OpenAPI documentation
- Infrastructure must be documented with diagrams
- Security controls must be clearly documented
...

File 2: `.kiro/steering/security-requirements.md`
``markdown

inclusion: fileMatch
fileMatchPattern: '*security*'

Security Requirements for MyCoolChatbot-KiroSpec Project

Authentication and Authorization
- Implement JWT-based authentication
- Use role-based access control (RBAC)
- Secure session management
- Multi-factor authentication support

Data Protection
- Encrypt all data in transit and at rest
- Implement proper input validation
- Follow GDPR compliance requirements
- Secure API endpoints with proper authentication

AWS Security
- Use AWS IAM for access control
- Implement CloudTrail for audit logging
- Use AWS Secrets Manager for sensitive data
- Configure security groups with minimal access
...

File 3: `.kiro/steering/aws-architecture.md`
``markdown

inclusion: fileMatch
fileMatchPattern: '*infrastructure*'

AWS Architecture Guidelines

Serverless Best Practices
- Use Lambda for compute when possible
- Implement proper error handling and retries
- Configure appropriate timeout and memory settings
- Use environment variables for configuration

API Gateway Configuration
- Implement proper CORS policies
- Use request validation
- Configure throttling and rate limiting
- Implement comprehensive logging

Bedrock Integration
- Use appropriate model selection
- Implement cost optimization strategies
- Handle API rate limits and errors
- Secure API keys and credentials
...

```

- Comprehensive Agent Ste

- Automated application of project standards and best practices
- Enhanced code quality through intelligent suggestions

### Exercise 2.5: Testing Kiro Features Integration

validate that all Kiro features work together effectively.

```

Test the integration of MCP Server, Agent Hooks, and Agent Steering.

Integration Test 1: MCP Server + Agent Steering

Ask Kiro: "Help me create an AWS Lambda function specification following our project standards"

- Verify that Kiro uses AWS documentation from MCP server
- Confirm that project standards from Agent Steering are applied
- Check that security requirements are automatically included

Integration Test 2: Agent Hooks + Specifications

1. Create a new specification file in the infrastructure folder
2. Save the file and verify the Documentation Sync hook triggers
3. Check that README.md and tasks.md are automatically updated
4. Confirm that related files are cross-referenced

Integration Test 3: Complete Workflow Test

Ask Kiro: "Create a comprehensive API Gateway specification for our chatbot, including security, monitoring, and deployment considerations"

- Verify MCP servers provide current AWS best practices
- Confirm Agent Steering applies security and architecture guidelines
- Check that Agent Hooks update documentation automatically
- Validate that all features work seamlessly together

Troubleshooting Common Issues:
```

- MCP server connection problems and solutions
- Agent Hook performance optimization
- Agent Steering file conflicts and resolution
- Integration debugging and monitoring

---

## Expected Outcome

- All Kiro features working together seamlessly
- Validated integration between MCP servers, Agent Hooks, and Agent Steering
- Troubleshooting knowledge for common issues

### Key Benefits of k

- After completing the Kiro Features Setup, you'll have:
1. **Enhanced Research Capabilities:** Real-time access to AWS documentation and
  2. **Automated Quality Assurance:** Agent Hooks that validate and maintain spec
  3. **Consistent Development:** Agent Steering that ensures adherence to project s

#### 4. Integrated Workflow: All features working together in the workshop studio

Event ends in 2 days 8 hours 11 minutes.

- The next section will guide you through structured requirements gathering techniques that leverage